

Lottoziehung in Python

6 aus 45 – Ziehung ohne Wiederholung und Statistik

Überblick: Was verlangt die Aufgabe?

Anforderung aus der Aufgabe	Wo im Code gelöst
Lottoziehung als Methode mit dem Zufallsgenerator	lottoziehung()
Keine Zahl darf zweimal gezogen werden	Topf-Liste, gezogene Kugel wird entfernt
Für alle 45 Zahlen nur 45 Zufallsaufrufe	Zufall wählt nur aus den <i>übrigen</i> Kugeln
Sechs Zahlen ziehen und ausgeben	Hauptprogramm, print(...)
Statistik als Methode, 1000 / 10000 / 100000 Ziehungen	lotto_statistik(n)
Dictionary: wie oft wurde welche Zahl gezogen	statistik_erstellen()
Statistikmethode nach jeder Ziehung aufrufen, Zähler erhöhen	statistik_aktualisieren()

Hinweis: Eine Funktion `random.getrand( )` gibt es in Python nicht. Gemeint ist das `random`-Modul. Wir verwenden `random.randint(a, b)`, das eine ganze Zufallszahl zwischen `a` und `b` liefert (beide Grenzen inklusive).

Schritt 0: Import und Konstanten

```
import random

MAX_ZAHL = 45      # Zahlen von 1 bis 45
ANZAHL_GEZOGEN = 6 # 6 Zahlen pro Ziehung
```

`import random` lädt Pythons eingebautes Modul für Zufallszahlen. Die beiden Konstanten stehen in Großbuchstaben, weil sie sich während des Programms nicht ändern. Der Vorteil: Will man später z. B. "6 aus 49" spielen, ändert man nur eine Zeile, statt überall im Code die Zahl 45 zu suchen.

Teil 1: Die Lottoziehung als Methode

```
def lottoziehung(anzahl=ANZAHL_GEZOGEN, max_zahl=MAX_ZAHL):
    topf = list(range(1, max_zahl + 1)) # alle Kugeln im Topf
    gezogen = []

    for _ in range(anzahl):
        index = random.randint(0, len(topf) - 1)
        zahl = topf[index]
        gezogen.append(zahl)

        topf[index] = topf[-1]
        topf.pop()

    return gezogen
```

1.1 Der Kopf der Methode

`def lottoziehung(anzahl=6, max_zahl=45)` definiert eine Funktion mit zwei Parametern. Durch die Standardwerte kann man sie einfach mit `lottoziehung()` aufrufen und bekommt 6 Zahlen. Man kann aber auch `lottoziehung(anzahl=45)` schreiben, um alle Kugeln zu ziehen – das brauchen wir später als Beweis.

1.2 Der Topf

`list(range(1, max_zahl + 1))` erzeugt die Liste `[1, 2, 3, ..., 45]`. Das ist wie die Lottotrommel mit allen Kugeln. Achtung: `range` schließt die obere Grenze *aus*, deshalb steht dort `+ 1`. `gezogen` ist eine leere Liste, in die die Gewinnzahlen kommen.

1.3 Warum nicht einfach `randint(1, 45)`?

Die naheliegende Idee wäre: Zufallszahl von 1 bis 45 ziehen und, falls sie schon gezogen wurde, nochmal würfeln. Das verbietet die Aufgabe. Bei der 45. Zahl wäre nur noch *eine* Zahl frei, und man müsste im Schnitt 45-mal würfeln, bis man genau sie erwischt. Für alle 45 Zahlen bräuchte man so rund 200 Zufallsaufrufe statt 45.

1.4 Die Lösung: nur aus den übrigen Kugeln wählen

Statt einer *Zahl* würfeln wir eine *Position* im Topf: `random.randint(0, len(topf) - 1)`. Weil der Topf nach jeder Ziehung um eine Kugel kleiner wird, liegt jede gewürfelte Position garantiert auf einer Kugel, die noch nicht gezogen wurde. Damit trifft **jeder** Zufallsaufruf, und es gibt nie einen Fehlversuch. Der Index geht von 0 bis Länge – 1, weil Listen in Python bei 0 anfangen zu zählen.

1.5 Der Tausch-Trick beim Entfernen

Die gezogene Kugel muss aus dem Topf. Dafür kopieren wir die **letzte** Kugel an die Stelle der gezogenen (`topf[index] = topf[-1]`) und werfen dann das letzte Element weg (`topf.pop()`). Die Reihenfolge im Topf ist egal, denn wir ziehen ja zufällig. Beispiel mit einem kleinen Topf aus 6 Kugeln:

Schritt	Topf vorher	Zufälliger Index	Gezogen	Topf nachher
1	[1, 2, 3, 4, 5, 6]	1	2	[1, 6, 3, 4, 5]
2	[1, 6, 3, 4, 5]	3	4	[1, 6, 3, 5]
3	[1, 6, 3, 5]	0	1	[5, 6, 3]
4	[5, 6, 3]	2	3	[5, 6]

Bei Schritt 4 war die gezogene Kugel selbst die letzte: Sie wird "mit sich selbst getauscht" und dann entfernt – das funktioniert also auch ohne Sonderfall. Man könnte auch `topf.pop(index)` schreiben, das ist genauso korrekt. Der Tausch-Trick ist nur schneller, weil Python beim Entfernen aus der Mitte alle nachfolgenden Elemente verschieben müsste.

1.6 Beweis: 45 Zahlen mit genau 45 Aufrufen

Die Schleife `for _ in range(anzahl)` läuft genau `anzahl`-mal, und in jedem Durchlauf steht genau ein `randint`. Bei 45 Zahlen sind das also exakt 45 Aufrufe. Das Programm prüft zusätzlich, dass keine Zahl doppelt vorkommt:

```
alle = lottoziehung(anzahl=45)
print("Alle 45 gezogen, keine doppelt:", len(set(alle)) == 45)
```

Ein `set` (Menge) speichert jeden Wert nur einmal. Hat das Set 45 Elemente, waren alle 45 gezogenen Zahlen verschieden. Ich habe zusätzlich die Aufrufe von `randint` mitgezählt: Ergebnis 45.

Merke: Der Unterstrich in `for _ in range(...)` ist ein üblicher Variablenname für "brauche ich nicht". Wir wollen die Schleife nur oft genug wiederholen, die Zählvariable selbst interessiert uns nicht.

1.7 Sechs Zahlen ziehen und ausgeben

```
zahlen = lottoziehung()
print("Die Lottozahlen lauten:", sorted(zahlen))
```

`sorted()` sortiert die Zahlen nur für die Anzeige, so wie sie auch im Fernsehen gezeigt werden. Beispielausgabe:

```
Die Lottozahlen lauten: [22, 36, 37, 39, 40, 45]
```

Teil 2: Die Statistik als Methode

Die Statistik besteht aus drei kleinen Methoden, die jeweils eine Aufgabe erledigen: Dictionary anlegen, Zähler erhöhen und viele Ziehungen durchführen. Dazu kommt eine Methode für die Ausgabe.

2.1 Das Dictionary erstellen

```
def statistik_erstellen(max_zahl=MAX_ZAHL):  
    return {zahl: 0 for zahl in range(1, max_zahl + 1)}
```

Ein Dictionary speichert Paare aus **Schlüssel** und **Wert**. Hier ist der Schlüssel die Lottozahl und der Wert, wie oft sie gezogen wurde. Die Schreibweise in geschweiften Klammern heißt *Dictionary Comprehension* und ist eine Kurzform für:

```
statistik = {}  
for zahl in range(1, 46):  
    statistik[zahl] = 0
```

Ergebnis: {1: 0, 2: 0, 3: 0, ..., 45: 0}. Alle Zähler starten bei 0.

2.2 Zähler nach jeder Ziehung erhöhen

```
def statistik_aktualisieren(statistik, ziehung):  
    for zahl in ziehung:  
        statistik[zahl] += 1
```

Diese Methode bekommt das Dictionary und die sechs Zahlen einer Ziehung. Für jede Zahl wird ihr Zähler um eins erhöht. `statistik[zahl] += 1` ist die Kurzform von `statistik[zahl] = statistik[zahl] + 1`.

Wichtig: Die Methode gibt nichts zurück (`return` fehlt) und muss es auch nicht. Ein Dictionary ist in Python *veränderbar*. Die Methode bekommt keine Kopie, sondern eine Referenz auf dasselbe Dictionary. Die Änderungen sind deshalb auch außerhalb der Methode sichtbar.

Ziehung	Gezogene Zahlen	Zähler danach (Auszug)
Start	-	{3: 0, 17: 0, 22: 0, ...}
1	[3, 17, 22, 30, 41, 44]	{3: 1, 17: 1, 22: 1, ...}
2	[5, 17, 22, 28, 33, 40]	{3: 1, 17: 2, 22: 2, ...}
3	[3, 9, 12, 17, 35, 45]	{3: 2, 17: 3, 22: 2, ...}

2.3 Viele Ziehungen durchführen

```
def lotto_statistik(anzahl_ziehungen):  
    statistik = statistik_erstellen()  
    for _ in range(anzahl_ziehungen):  
        ziehung = lottoziehung()  
        statistik_aktualisieren(statistik, ziehung)  
    return statistik
```

Hier kommt alles zusammen. Zuerst wird ein leeres Dictionary angelegt. Dann wird so oft gezogen wie gewünscht, und genau wie in der Aufgabe verlangt, wird **nach jeder einzelnen Ziehung** die Statistikmethode aufgerufen. Am Ende wird das fertige Dictionary zurückgegeben.

2.4 Die Statistik ausgeben

```
def statistik_ausgeben(statistik, anzahl_ziehungen):
    erwartet = anzahl_ziehungen * ANZAHL_GEZOGEN / MAX_ZAHL
    print(f"\n=== Statistik nach {anzahl_ziehungen} Ziehungen ===")
    print(f"Erwartet pro Zahl: ca. {erwartet:.1f} mal")
    for zahl, anzahl in statistik.items():
        abweichung = (anzahl - erwartet) / erwartet * 100
        print(f"Zahl {zahl:2d}: {anzahl:8d} mal ({abweichung:+6.2f} %)")
    haeufigste = max(statistik, key=statistik.get)
    seltenste = min(statistik, key=statistik.get)
    print(f"Am haeufigsten: {haeufigste} ({statistik[haeufigste]} mal)")
    print(f"Am seltensten: {seltenste} ({statistik[seltenste]} mal)")
```

Erwartungswert: Pro Ziehung werden 6 von 45 Zahlen gezogen. Jede Zahl hat also die Chance $6/45 = 13,3\%$. Bei 1000 Ziehungen sollte jede Zahl ungefähr $1000 \cdot 6 / 45 = 133$ -mal vorkommen.

items(): liefert alle Paare (Zahl, Anzahl), sodass man beide gleichzeitig in der Schleife hat.

f-Strings: Das f vor dem Text erlaubt Variablen in geschweiften Klammern. Hinter dem Doppelpunkt steht die Formatierung: `:2d` = ganze Zahl, 2 Stellen breit; `:.1f` = eine Nachkommastelle; `:+6.2f` = immer mit Vorzeichen, 2 Nachkommastellen. So stehen die Spalten sauber untereinander.

max mit key: `max(statistik, key=statistik.get)` sucht den Schlüssel, dessen *Wert* am größten ist, also die am häufigsten gezogene Zahl. Ohne key würde einfach die 45 herauskommen, weil sie der größte Schlüssel ist.

Teil 3: Das Hauptprogramm

```
if __name__ == "__main__":
    zahlen = lottoziehung()
    print("Die Lottozahlen lauten:", sorted(zahlen))

    alle = lottoziehung(anzahl=45)
    print("Alle 45 gezogen, keine doppelt:", len(set(alle)) == 45)

    for n in [1000, 10000, 100000]:
        stat = lotto_statistik(n)
        statistik_ausgeben(stat, n)
```

if __name__ == "__main__": sorgt dafür, dass dieser Teil nur läuft, wenn man die Datei direkt startet. Importiert man sie in ein anderes Programm, stehen nur die Methoden zur Verfügung, ohne dass sofort gezogen wird. Die Schleife über [1000, 10000, 100000] setzt das "usw." aus der Aufgabe um. Wer mehr will, hängt einfach 1000000 an die Liste (dauert ein paar Sekunden).

Was zeigt die Statistik?

Ein Testlauf ergab folgende Werte. Die letzte Spalte zeigt, wie weit die "ungerechteste" Zahl vom Erwartungswert abwich:

Ziehungen	Erwartet pro Zahl	Seltenste Zahl	Häufigste Zahl	Größte Abweichung
1.000	133	108-mal	164-mal	23,0 %
10.000	1.333	1.238-mal	1.407-mal	7,2 %
100.000	13.333	13.137-mal	13.511-mal	1,5 %
1.000.000	133.333	132.725-mal	134.125-mal	0,6 %

Bei wenigen Ziehungen schwanken die Zahlen stark, manche kommen fast ein Viertel öfter oder seltener vor. Je mehr Ziehungen, desto näher liegen alle Zahlen am Erwartungswert. Das ist das **Gesetz der großen Zahlen**. Es zeigt auch, dass der Algorithmus fair ist: Keine Zahl wird systematisch bevorzugt. Deine eigenen Werte werden etwas anders aussehen, weil jeder Lauf andere Zufallszahlen erzeugt.

Typischer Denkfehler: Auch wenn bei 1000 Ziehungen eine Zahl "seltener" war, hat sie bei der nächsten Ziehung keine höhere Chance. Der Zufallsgenerator hat kein Gedächtnis. Jede Ziehung startet mit einem vollen Topf.

Der komplette Code

```
import random
MAX_ZAHL = 45          # Zahlen von 1 bis 45
ANZAHL_GEZOGEN = 6     # 6 Zahlen pro Ziehung
# -----
# Teil 1: Lottoziehung als Methode
# -----
def lottoziehung(anzahl=ANZAHL_GEZOGEN, max_zahl=MAX_ZAHL):
    """Zieht 'anzahl' verschiedene Zahlen aus 1..max_zahl.
    Pro gezogener Zahl wird der Zufallsgenerator genau EINMAL benutzt."""
    topf = list(range(1, max_zahl + 1)) # alle Kugeln im Topf
    gezogen = []
    for _ in range(anzahl):
        # zufaelligen Index nur aus den NOCH VORHANDENEN Kugeln waehlen
        index = random.randint(0, len(topf) - 1)
        zahl = topf[index]
        gezogen.append(zahl)
        # gezogene Kugel entfernen: letzte Kugel an ihre Stelle setzen
        topf[index] = topf[-1]
        topf.pop()
    return gezogen
# -----
# Teil 2: Statistik
# -----
def statistik_erstellen(max_zahl=MAX_ZAHL):
    """Dictionary mit allen Zahlen als Schluessel und Zaehler 0."""
    return {zahl: 0 for zahl in range(1, max_zahl + 1)}
def statistik_aktualisieren(statistik, ziehung):
    """Wird nach JEDER Ziehung aufgerufen und erhoeht die Zaehler."""
    for zahl in ziehung:
        statistik[zahl] += 1
def lotto_statistik(anzahl_ziehungen):
    """Macht 'anzahl_ziehungen' Ziehungen und zaehlt mit."""
    statistik = statistik_erstellen()
    for _ in range(anzahl_ziehungen):
        ziehung = lottoziehung()
        statistik_aktualisieren(statistik, ziehung)
    return statistik
def statistik_ausgeben(statistik, anzahl_ziehungen):
    erwartet = anzahl_ziehungen * ANZAHL_GEZOGEN / MAX_ZAHL
    print(f"\n== Statistik nach {anzahl_ziehungen} Ziehungen ==")
    print(f"Erwartet pro Zahl: ca. {erwartet:.1f} mal")
    for zahl, anzahl in statistik.items():
        abweichung = (anzahl - erwartet) / erwartet * 100
        print(f"Zahl {zahl:2d}: {anzahl:8d} mal ({abweichung:+6.2f} %)")
    haeufigste = max(statistik, key=statistik.get)
    seltenste = min(statistik, key=statistik.get)
    print(f"Am haeufigsten: {haeufigste} ({statistik[haeufigste]} mal)")
    print(f"Am seltensten: {seltenste} ({statistik[seltenste]} mal)")
# -----
# Hauptprogramm
# -----
if __name__ == "__main__":
    # 1) Eine Ziehung am Bildschirm ausgeben
    zahlen = lottoziehung()
    print("Die Lottozahlen lauten:", sorted(zahlen))
    # 2) Beweis: alle 45 Zahlen ziehen -> keine doppelt
    alle = lottoziehung(anzahl=45)
    print("Alle 45 gezogen, keine doppelt:", len(set(alle)) == 45)
    # 3) Statistik fuer 1000, 10000, 100000 Ziehungen
    for n in [1000, 10000, 100000]:
        stat = lotto_statistik(n)
        statistik_ausgeben(stat, n)
```